FICHE REPÈRE

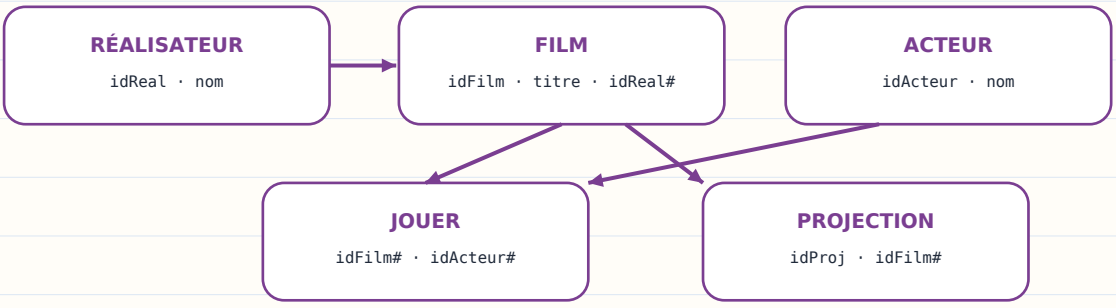
SQL pour la base cinéma

Guide visuel du corrigé complet · 70 questions · MariaDB 10.11 · 4e SI

SQL permet de créer une base relationnelle, de manipuler ses données et d'interroger plusieurs tables.



SCHÉMA RELATIONNEL





À retenir : Les clés primaires identifient les lignes ; les clés étrangères relient les tables. Le corrigé intégral de 28 pages est joint à la fin de ce guide.

PARTIE 1 · Q1–Q5

Construire la structure

LDD · CREATE, ALTER, DROP : définir d'abord tables, clés et règles.

Q1 · Créer les cinq tables

REALISATEUR, FILM, ACTEUR, JOUER et PROJECTION. Les clés composées restent utiles pour JOUER.

```
PRIMARY KEY (idFilm, idActeur)
FOREIGN KEY (idFilm) REFERENCES film(idFilm)
CHECK (duree > 0) · CHECK (annee >= 1900)
```

Q2 · Ajouter une colonne

ALTER ajoute une nouvelle colonne sans reconstruire toute la table.

```
ALTER TABLE acteur
ADD email VARCHAR(50);
```

Q3 · Modifier une colonne

MODIFY redéfinit la colonne : il faut répéter NOT NULL pour conserver la contrainte.

```
ALTER TABLE film
MODIFY titre VARCHAR(100) NOT NULL;
```

Q4 · Supprimer une colonne

DROP COLUMN enlève la colonne email.

```
ALTER TABLE acteur
DROP COLUMN email;
```

Q5 · Recréer une table

Supprimer puis recréer PROJECTION avec sa clé étrangère vers FILM.

```
DROP TABLE projection;
CREATE TABLE projection (...);
```

Lire le schéma

PK = identifiant unique. FK = référence vers une autre table. Les colonnes de la relation JOUER sont reliées à FILM et ACTEUR.

```
REALISATEUR 1 ── n FILM
FILM n ── n ACTEUR via JOUER
FILM 1 ── n PROJECTION
```

À retenir : Avant un ALTER TABLE, vérifie les données existantes. NOT NULL ou CHECK peuvent refuser la modification si des lignes les contredisent.

PARTIE 2 · Q6–Q9

Installer et modifier les données

LMD · INSERT et UPDATE : commencer par un jeu d’essai cohérent.

Q6 · Charger le jeu de cinéma

3 réalisateurs · 6 films · 6 acteurs · au moins 10 rôles · 9 projections. Les clés doivent exister avant d’insérer les lignes enfants.

```
INSERT INTO realisateur VALUES (...);
INSERT INTO film (titre, genre, idReal) VALUES (...);
INSERT INTO jouer VALUES (idFilm, idActeur, ...);
```

ORDRE CONSEILLÉ : parents d’abord, enfants ensuite



Q7 · INSERT partiel

On ne fournit que les colonnes utiles ; les autres restent NULL ou prennent DEFAULT.

```
INSERT INTO film (titre, genre, idReal)
VALUES ('Tenet', 'Action', 1);
```

Q8 · UPDATE ciblé

Toujours associer SET à un WHERE précis pour éviter de modifier toute la table.

```
UPDATE film SET duree = 148
WHERE titre = 'Inception';
```

Q9 · Calculer une hausse

Augmenter de 10 % seulement les cachets inférieurs à 5 000. La formule multiplie par 1,10.

```
UPDATE jouer
SET cachet = cachet * 1.10
WHERE cachet < 5000;
```

À retenir : Après chaque INSERT ou UPDATE, exécute un SELECT de vérification : le corrigé donne les résultats en tenant compte de l’ordre des opérations.

PARTIE 2 · Q10–Q15

Mettre à jour et supprimer

Quand la modification dépend d’autres tables, relier les lignes ou utiliser une sous-requête.

Q10 · UPDATE avec jointure

Augmenter de 20 % les cachets des rôles de films Action réalisés par Nolan.

```
UPDATE jouer j, film f, realisateur r
SET j.cachet = j.cachet * 1.20
WHERE j.idFilm=f.idFilm AND f.idReal=r.idReal;
```

Q11 · UPDATE avec sous-requête

Mettre le budget à 0 pour les films sans acteur.

```
UPDATE film SET budget = 0
WHERE idFilm NOT IN (SELECT idFilm FROM jouer);
```

Q12 · Comparer à une moyenne

Passer en Blockbuster les films dépassant la moyenne des films Action.

```
UPDATE film SET genre = 'Blockbuster'
WHERE budget > (SELECT AVG(budget) ...);
```

Q13 · DELETE avec filtre

Supprimer les projections de moins de 10 spectateurs.

```
DELETE FROM projection
WHERE nbSpectateurs < 10;
```

Q14 · DELETE avec jointure

Retirer de JOUER les rôles des films de Scorsese.

```
DELETE j FROM jouer j, film f, realisateur r
WHERE j.idFilm=f.idFilm AND f.idReal=r.idReal;
```

Q15 · DELETE sans correspondance

Supprimer les acteurs qui ne figurent dans aucun rôle.

```
DELETE FROM acteur
WHERE idActeur NOT IN (SELECT idActeur FROM jouer);
```

À retenir : Évite d’omettre WHERE dans UPDATE/DELETE. Pour les relations multi-tables, vérifie d’abord les identifiants avec un SELECT.

PARTIE 3 · Q16–Q29

Lire, filtrer et relier

SELECT : transformer une demande en projection, condition, tri, puis jointure.

Q16 Colonnes + alias

SELECT titre, duree AS dureeMin

Q17 Filtre exact

WHERE genre = 'Action'

Q18 Intervalle inclusif

BETWEEN 2008 AND 2010

Q19 Plusieurs valeurs

IN ('Thriller', 'Comédie')

Q20 Début de texte

LIKE 'I%' · % = suite quelconque

Q21 Valeurs absentes

IS NULL · jamais = NULL

Q22 Deux critères

duree > 140 AND genre <> ...

Q23 Sans doublons

SELECT DISTINCT genre

Q24 Filtrer + trier

WHERE nationalite=... ORDER BY nom

Q25 Tri à deux niveaux

ORDER BY annee DESC, titre ASC

Q26 Top 3

ORDER BY duree DESC LIMIT 3

Q27 Relier film/réalisateur

f.idReal = r.idReal

Q28 Trois tables

acteur → jouer → film

Q29 Jointure + filtre

réalisateur Nolan + cachet > 10000

Patron de jointure

Relie les identifiants avant de filtrer le résultat.

```
SELECT f.titre, r.nom
FROM film f, realisateur r
WHERE f.idReal = r.idReal;
```

À retenir : Construis la requête de gauche à droite : quelles colonnes ? quelles tables ? quelles relations ? quelles conditions ? quel tri ?

PARTIE 3 · Q30–Q42

Résumer et comparer

GROUP BY, HAVING et sous-requêtes : passer des lignes individuelles aux réponses calculées.

Q30 Compter par genre

COUNT(*) + GROUP BY genre

Q31 Moyenne de cachets

AVG(cachet), arrondie à 2 décimales

Q32 Réalisateurs prolifiques

GROUP BY réalisateur · HAVING COUNT(*) > 3

Q33 Total par acteur

SUM(cachet) · HAVING total > 15000

Q34 Budget maximal

MAX(budget) par genre/réalisateur · HAVING > 100 M

Q35 Auto-jointure

idParrain relie un acteur à un autre

Q36 Paires de films

même réalisateur · idFilm1 < idFilm2

Q37 Dates identiques

auto-jointure sur dateNaiss

Q38 Au-dessus de la moyenne

duree > (SELECT AVG(duree)...)

Q39 Au moins un film comédie

IN (SELECT idFilm...)

Q40 Aucun film

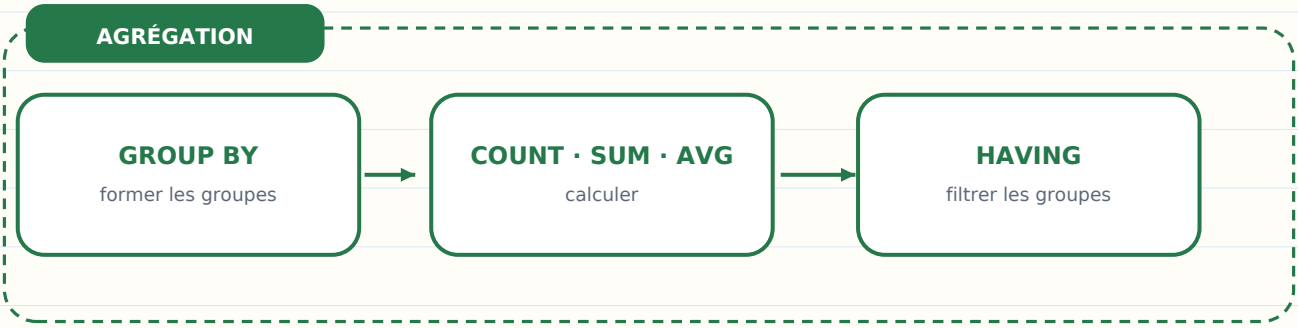
NOT IN / NOT EXISTS

Q41 Cachet maximal

MAX(cachet) dans Inception

Q42 Compter les acteurs

comparaison au nombre moyen par film



À retenir : WHERE filtre les lignes avant le regroupement ; HAVING filtre les groupes après le calcul.

PARTIE 4 · Q43–Q49

Travailler avec les dates

Les fonctions extraient, comparent et affichent les dates au bon niveau de précision.

Q43 · Année de naissance

Extraire l'année avant de comparer.

```
YEAR(dateNaiss) = 1980
```

Q44 · Calculer un âge

L'âge dépend de la date de référence.

```
TIMESTAMPDIFF(YEAR, dateNaiss, CURDATE())
```

Q45 · Période de projections

La borne finale est incluse.

```
BETWEEN 01/11/2026 AND 31/12/2026
```

Q46 · Compter par mois

Regrouper les projections par numéro de mois.

```
MONTH(dateProj) + GROUP BY
```

Q47 · Naissances en juillet

Le mois de juillet vaut 7.

```
MONTH(dateNaiss) = 7
```

Q48 · 30 derniers jours

Comparer une date à une borne mobile.

```
CURDATE() et DATE_SUB(..., INTERVAL 30 DAY)
```

Q49 · Nom du jour

La langue de session règle l'affichage des jours.

```
DAYNAME(dateProj)
```

DATE **YEAR / MONTH** **INTERVALLE**

À retenir : Le corrigé fixe CURDATE() au 1er novembre 2026. Une autre date courante peut donc produire des âges ou périodes différents.

PARTIE 5 · Q50–Q54

Prévoir les contraintes

DEFAULT, NULL et clés étrangères : raisonner sur la valeur et sur la relation.

Q50 · Ajouter DEFAULT

Les anciennes lignes reçoivent une valeur compatible.

```
paysOrigine NOT NULL DEFAULT 'Inconnue'
```

Q51 · Changer DEFAULT

Ne modifie pas les valeurs déjà stockées.

```
DEFAULT 'Tunisie'
```

Q52 · Rendre obligatoire

Une valeur NULL existante empêche la contrainte.

```
corriger les NULL avant MODIFY ... NOT NULL
```

Q53 · ON UPDATE CASCADE

La clé enfant suit automatiquement la clé parent.

```
idFilm : 7 → 70
```

Q54 · ON DELETE CASCADE

Les lignes enfants qui le référencent disparaissent.

```
DELETE film 70
```

Règle mentale · parent → enfant

UPDATE CASCADE propage une nouvelle clé. DELETE CASCADE supprime les lignes dépendantes. Sans CASCADE, une ligne parent référencée ne peut généralement pas être supprimée.

À retenir : DEFAULT s’applique lorsqu’une insertion omet la colonne. MODIFY redéfinit la colonne : répète toutes les contraintes à conserver.

PARTIE 6 · Q55–Q65

Prévoir le résultat

Ici, on lit les données et la requête sans l'exécuter : compter les lignes qui survivent.

Q	IDÉE	RÉSULTAT CLÉ
55	Jointure après cascade	1 ligne : projection 102 → Inception
56	COUNT avec filtre	3 cachets > 10 000
57	COUNT(*) / COUNT(duree)	7 films · 6 durées non NULL
58	AVG(duree)	142,8333 : le NULL est ignoré
59	NULL et <>	Film test absent : NULL <> 148 est UNKNOWN
60	GROUP BY acteur	Seuls les acteurs présents dans JOUER apparaissent
61	HAVING sur spectateurs	Inception 205 · Interstellar 260
62	Statistiques par salle	Salle 1: 4 / 150 · Salle 2: 2 / 120 · Salle 3: 1 / 70
63	Auto-jointure parrain	Bale et McConaughey → DiCaprio
64	Mois de naissance 11	DiCaprio, Bale, McConaughey, Hathaway
65	SUM sur Inception	29 350,00



À retenir : Pour prévoir une sortie, applique dans l'ordre : WHERE → jointure → GROUP BY → HAVING → ORDER BY. Vérifie aussi les NULL.

PARTIE 7 · Q66–Q70

Comprendre réussite et erreur

Une erreur SQL révèle souvent la règle de structure qui protège la cohérence.

Q66 · Clé primaire doublée

ÉCHEC · 1062 · Le couple (idFilm, idActeur) existe déjà.

Q67 · CHECK sur l’année

ÉCHEC · 4025 · 1850 viole CHECK (annee >= 1900).

Q68 · Clé étrangère absente

ÉCHEC · 1452 · Le film 99 n’existe pas dans FILM.

Q69 · Supprimer un parent référencé

ÉCHEC · 1451 · OSS 117 référence le réalisateur 3 ; pas de ON DELETE CASCADE.

Q70 · Supprimer une ligne sans dépendance

RÉUSSITE · Le film 8 est sans référence ; il reste 6 films.

Méthode en 4 questions

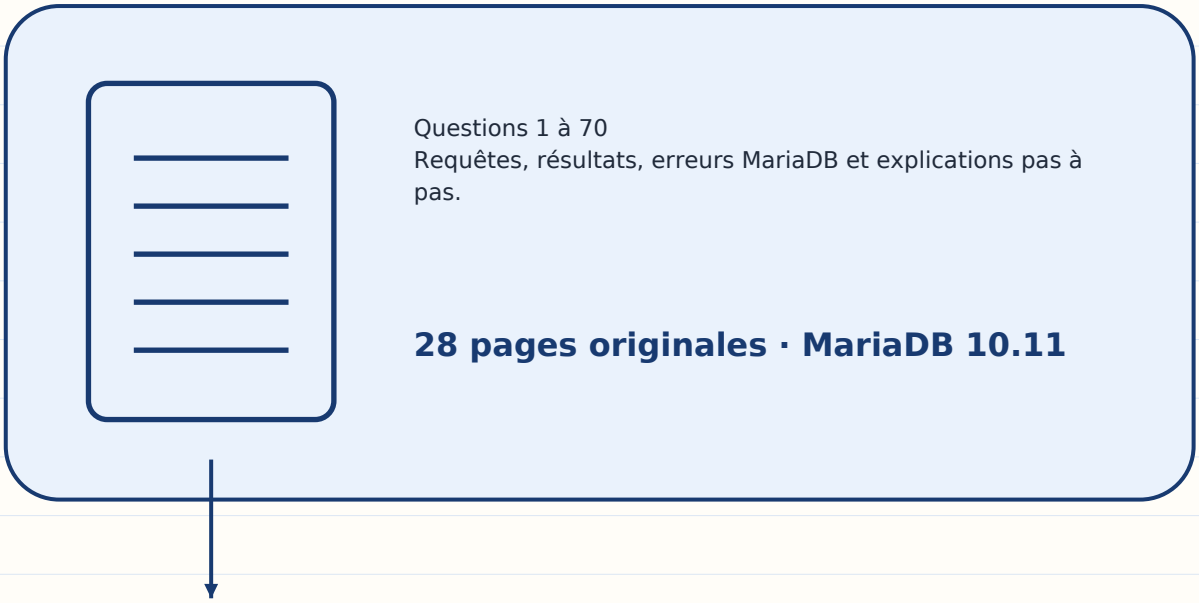
1. La clé primaire existe-t-elle déjà ? 2. La valeur respecte-t-elle CHECK / NOT NULL ?
3. La clé étrangère pointe-t-elle vers un parent existant ? 4. Une ligne enfant empêche-t-elle DELETE ?

À retenir : Synthèse : DEFAULT remplit une omission ; NULL se teste avec IS NULL ; les contraintes contrôlent chaque insertion, mise à jour et suppression.

ANNEXE INTÉGRALE

Corrigé officiel complet

Les fiches visuelles résument les 70 questions. Le corrigé détaillé original suit dans son intégralité.



Questions 1 à 70
Requêtes, résultats, erreurs MariaDB et explications pas à pas.

28 pages originales · MariaDB 10.11

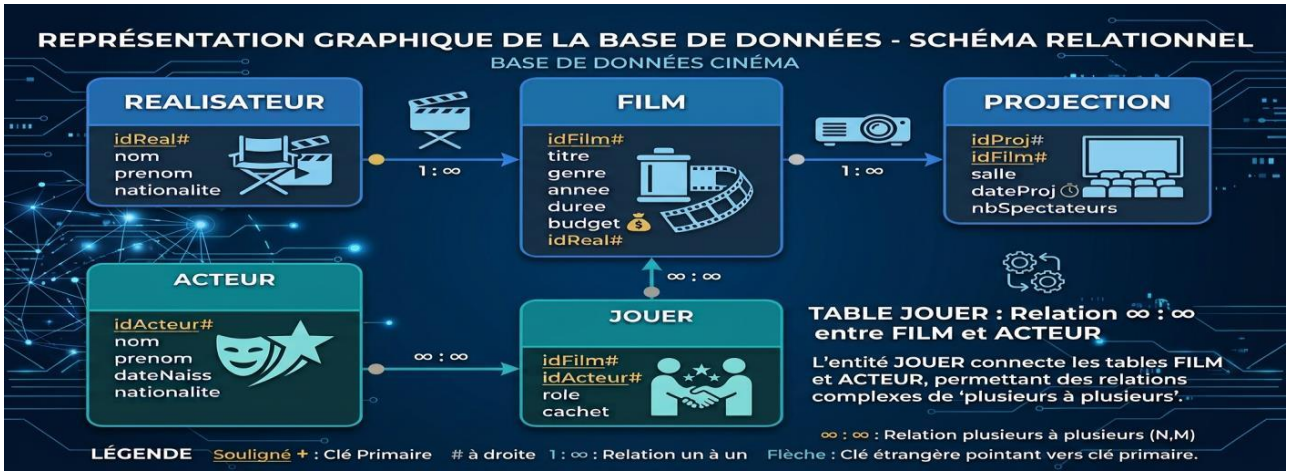
À retenir : Après les fiches, retrouvez les commandes et résultats du corrigé source sans aucune modification.

Corrigé : Série de TD SQL (thème Cinéma)

Enseignant : Aymen Essouyah | Lycée Rafèha | 4ème SI

Schéma de la base cinema

- **REALISATEUR** (idReal, nom, prenom, nationalite)
- **FILM** (idFilm, titre, genre, annee, duree, budget, idReal#)
- **ACTEUR** (idActeur, nom, prenom, dateNaiss, nationalite)
- **JOUER** (idFilm#, idActeur#, role, cachet)
- **PROJECTION** (idProj, idFilm#, salle, dateProj, nbSpectateurs)



> **Remarques.** Les requêtes ont été exécutées dans l'ordre sur MariaDB 10.11 (compatible MySQL) : chaque résultat tient compte des modifications précédentes. Pour `CURDATE()`, les résultats sont donnés pour la date du 01/11/2026 (pour retrouver exactement ces résultats un autre jour : `SET timestamp = UNIX_TIMESTAMP('2026-11-01 10:00:00');`). Quand une requête peut s'écrire autrement, la ou les **autres écritures** sont données dans un encadré vert ; chacune a été exécutée et donne le même résultat. Les jointures s'écrivent `table1.x = table2.x` (sans `JOIN ... ON`).

Partie 1 : LDD (Définition) : CREATE, ALTER, DROP

1.1 Création de tables (CREATE TABLE) : clés primaires, clés étrangères, CHECK

Question 1

Créer les tables REALISATEUR, FILM, ACTEUR, JOUER et PROJECTION avec leurs clés primaires et étrangères. `duree` doit être > 0, `annee` ≥ 1900 et `cachet` ≥ 0. (`idParrain` sera ajoutée à la question 35.)

```
CREATE TABLE realisateur (
  idReal      INT PRIMARY KEY,
  nom         VARCHAR(30) NOT NULL,
  prenom      VARCHAR(30),
  nationalite VARCHAR(30)
);
CREATE TABLE acteur (
  idActeur    INT PRIMARY KEY,
  nom         VARCHAR(30) NOT NULL,
  prenom      VARCHAR(30),
  dateNaiss   DATE,
  nationalite VARCHAR(30)
);
CREATE TABLE film (
  idFilm INT AUTO_INCREMENT PRIMARY KEY,
  titre  VARCHAR(50) NOT NULL,
```

```
  genre  VARCHAR(30),
  annee  INT CHECK (annee >= 1900),
  duree  INT CHECK (duree > 0),
  budget BIGINT,
  idReal INT,
  FOREIGN KEY (idReal) REFERENCES realisateur(idReal)
);
CREATE TABLE jouer (
  idFilm   INT,
  idActeur INT,
  role     VARCHAR(40),
  cachet   DECIMAL(10,2) CHECK (cachet >= 0),
  PRIMARY KEY (idFilm, idActeur),
  FOREIGN KEY (idFilm) REFERENCES film(idFilm),
  FOREIGN KEY (idActeur) REFERENCES acteur(idActeur)
);
CREATE TABLE projection (
  idProj        INT PRIMARY KEY,
  idFilm        INT,
  salle         VARCHAR(20),
  dateProj      DATE,
  nbSpectateurs INT,
  FOREIGN KEY (idFilm) REFERENCES film(idFilm)
);
-- Vérification
SHOW TABLES;
```

Résultat

Tables_in_cinema
acteur
film
jouer
projection
realisateur

Autre écriture : Contraintes nommées, déclarées au niveau de la table

```
CREATE TABLE realisateur (
  idReal      INT,
  nom         VARCHAR(30) NOT NULL,
  prenom      VARCHAR(30),
  nationalite VARCHAR(30),
  CONSTRAINT pk_realisateur PRIMARY KEY (idReal)
);
CREATE TABLE acteur (
  idActeur    INT,
  nom         VARCHAR(30) NOT NULL,
  prenom      VARCHAR(30),
  dateNaiss   DATE,
  nationalite VARCHAR(30),
  CONSTRAINT pk_acteur PRIMARY KEY (idActeur)
);
CREATE TABLE film (
  idFilm INT AUTO_INCREMENT,
  titre  VARCHAR(50) NOT NULL,
  genre  VARCHAR(30),
  annee  INT,
  duree  INT,
  budget BIGINT,
  idReal INT,
  CONSTRAINT pk_film PRIMARY KEY (idFilm),
  CONSTRAINT fk_film_real FOREIGN KEY (idReal) REFERENCES realisateur(idReal),
  CONSTRAINT ck_annee CHECK (annee >= 1900),
  CONSTRAINT ck_duree CHECK (duree > 0)
);
CREATE TABLE jouer (
  idFilm   INT,
```

```
  idActeur INT,
  role     VARCHAR(40),
  cachet   DECIMAL(10,2),
  CONSTRAINT pk_jouer PRIMARY KEY (idFilm, idActeur),
  CONSTRAINT fk_jouer_film FOREIGN KEY (idFilm) REFERENCES film(idFilm),
  CONSTRAINT fk_jouer_acteur FOREIGN KEY (idActeur) REFERENCES acteur(idActeur),
  CONSTRAINT ck_cachet CHECK (cachet >= 0)
);
CREATE TABLE projection (
  idProj        INT,
  idFilm        INT,
  salle         VARCHAR(20),
  dateProj      DATE,
  nbSpectateurs INT,
  CONSTRAINT pk_projection PRIMARY KEY (idProj),
  CONSTRAINT fk_proj_film FOREIGN KEY (idFilm) REFERENCES film(idFilm)
);
```

1.2 Modification de la structure (ALTER TABLE) : ADD, MODIFY, DROP COLUMN

Question 2

Ajouter à ACTEUR une colonne `email` (VARCHAR 50).

```
ALTER TABLE acteur ADD email VARCHAR(50);
-- Vérification
DESCRIBE acteur;
```

Résultat

Field	Type	Null	Key	Default	Extra
idActeur	int(11)	NO	PRI	*NULL*	
nom	varchar(30)	NO		*NULL*	
prenom	varchar(30)	YES		*NULL*	
dateNaiss	date	YES		*NULL*	
nationalite	varchar(30)	YES		*NULL*	
email	varchar(50)	YES		*NULL*	

Question 3

Modifier le type de `titre` en VARCHAR(100) (sans perdre son `NOT NULL`) et rendre la colonne `genre` NOT NULL.

```
ALTER TABLE film MODIFY titre VARCHAR(100) NOT NULL;
ALTER TABLE film MODIFY genre VARCHAR(30) NOT NULL;
-- Vérification
DESCRIBE film;
```

Résultat

Field	Type	Null	Key	Default	Extra
idFilm	int(11)	NO	PRI	*NULL*	auto_increment
titre	varchar(100)	NO		*NULL*	
genre	varchar(30)	NO		*NULL*	
annee	int(11)	YES		*NULL*	
duree	int(11)	YES		*NULL*	
budget	bigint(20)	YES		*NULL*	
idReal	int(11)	YES	MUL	*NULL*	

Attention : `MODIFY` redéfinit entièrement la colonne. Écrire `MODIFY titre VARCHAR(100)` sans `NOT NULL` ferait disparaître la contrainte `NOT NULL` posée à la question 1 : il faut la répéter.

Autre écriture : Une seule instruction ALTER TABLE

```
ALTER TABLE film
  MODIFY titre VARCHAR(100) NOT NULL,
  MODIFY genre VARCHAR(30) NOT NULL;
```

Question 4

Supprimer la colonne `email`.

```
ALTER TABLE acteur DROP COLUMN email;
-- Vérification
DESCRIBE acteur;
```

Résultat

Field	Type	Null	Key	Default	Extra
idActeur	int(11)	NO	PRI	*NULL*	
nom	varchar(30)	NO		*NULL*	
prenom	varchar(30)	YES		*NULL*	
dateNaiss	date	YES		*NULL*	
nationalite	varchar(30)	YES		*NULL*	

1.3 Suppression d'une table (DROP TABLE)

Question 5

Supprimer la table PROJECTION, puis la recréer.

```
DROP TABLE projection;
CREATE TABLE projection (
  idProj        INT PRIMARY KEY,
  idFilm        INT,
  salle         VARCHAR(20),
  dateProj      DATE,
  nbSpectateurs INT,
  FOREIGN KEY (idFilm) REFERENCES film(idFilm)
);
-- Vérification
SHOW TABLES;
```

Résultat

Tables_in_cinema
acteur
film
jouer
projection
realisateur

Partie 2 : LMD (Manipulation) : insertion, modification, suppression

2.1 Insertion (INSERT)

Question 6

Insert : Insérer 3 réalisateurs, 6 films, 6 acteurs et au moins 10 lignes dans JOUER (+ 9 projections). Le cachet est exprimé en milliers de dollars ; distribution et dates de naissance adaptées pour les besoins de l'exercice.

```
INSERT INTO realisateur VALUES
(1,'Nolan','Christopher','Britannique'),
(2,'Scorsese','Martin','Américaine'),
(3,'Hazanavicius','Michel','Française');
INSERT INTO acteur VALUES
(1,'DiCaprio','Leonardo','1974-11-11','Américaine'),
(2,'Bale','Christian','1974-11-11','Britannique'),
(3,'Ledger','Heath','1979-04-04','Australienne'),
(4,'Dujardin','Jean','1972-06-19','Française'),
(5,'McConaughey','Matthew','1969-11-04','Américaine'),
(6,'Ruffalo','Mark','1967-11-22','Américaine');
INSERT INTO film (idFilm, titre, genre, annee, duree, budget, idReal) VALUES
(1,'Inception','Science-fiction',2010,140,160000000,1),
(2,'Interstellar','Science-fiction',2014,169,165000000,1),
(3,'The Dark Knight','Action',2008,152,185000000,1),
(4,'Les Infiltrés','Action',2006,151,90000000,2),
(5,'Shutter Island','Thriller',2010,138,80000000,2),
(6,'OSS 117','Comédie',2006,99,14000000,3);
INSERT INTO jouer VALUES
(1,1,'Dom Cobb',20000), (1,4,'Eames',4000), (1,3,'Arthur',4500),
(2,5,'Cooper',15000),
(3,2,'Bruce Wayne',12000), (3,3,'Joker',3000),
(4,1,'Billy Costigan',25000),
(5,1,'Teddy Daniels',30000), (5,6,'Chuck Aule',2500),
(6,4,'Hubert Bonisseur',1500), (6,2,'Agent Moreau',900);
INSERT INTO projection VALUES
(1,1,'Salle 1','2026-09-20',85),
(2,1,'Salle 2','2026-10-03',120),
(3,2,'Salle 1','2026-10-10',150),
(4,3,'Salle 3','2026-10-17',8),
(5,3,'Salle 1','2026-10-24',95),
(6,5,'Salle 2','2026-10-31',60),
(7,6,'Salle 3','2026-11-15',70),
(8,2,'Salle 1','2026-12-05',110),
(9,4,'Salle 2','2026-10-05',5);
-- Vérification
SELECT * FROM film;
SELECT * FROM jouer ORDER BY idFilm, idActeur;
```

Résultat

idFilm	titre	genre	annee	duree	budget	idReal
1	Inception	Science-fiction	2010	140	160000000	1
2	Interstellar	Science-fiction	2014	169	165000000	1
3	The Dark Knight	Action	2008	152	185000000	1
4	Les Infiltrés	Action	2006	151	90000000	2
5	Shutter Island	Thriller	2010	138	80000000	2
6	OSS 117	Comédie	2006	99	14000000	3

idFilm	idActeur	role	cachet
1	1	Dom Cobb	20000.00
1	3	Arthur	4500.00
1	4	Eames	4000.00
2	5	Cooper	15000.00
3	2	Bruce Wayne	12000.00
3	3	Joker	3000.00
4	1	Billy Costigan	25000.00
5	1	Teddy Daniels	30000.00
5	6	Chuck Aule	2500.00
6	2	Agent Moreau	900.00
6	4	Hubert Bonisseur	1500.00

Question 7

Insérer un nouveau film en ne renseignant que `titre`, `genre` et `idReal`.

```
INSERT INTO film (titre, genre, idReal) VALUES ('Tenet', 'Action', 1);
-- Vérification
SELECT * FROM film WHERE titre = 'Tenet';
```

Résultat

idFilm	titre	genre	annee	duree	budget	idReal
7	Tenet	Action	*NULL*	*NULL*	*NULL*	1

Autre écriture : Syntaxe INSERT ... SET

```
INSERT INTO film SET titre = 'Tenet', genre = 'Action', idReal = 1;
```

2.2 Modification (UPDATE) : simple, avec jointure, avec sous-requête

Question 8

Update : Mettre la durée du film « Inception » à 148 minutes.

```
UPDATE film SET duree = 148 WHERE titre = 'Inception';
-- Vérification
SELECT titre, duree FROM film WHERE titre = 'Inception';
```

Résultat

titre	duree
Inception	148

Question 9

Augmenter de 10 % le cachet de tous les rôles dont le cachet est inférieur à 5000.

```
UPDATE jouer SET cachet = cachet * 1.10 WHERE cachet < 5000;
-- Vérification
SELECT * FROM jouer ORDER BY idFilm, idActeur;
```

Résultat

idFilm	idActeur	role	cachet
1	1	Dom Cobb	20000.00
1	3	Arthur	4950.00
1	4	Eames	4400.00
2	5	Cooper	15000.00
3	2	Bruce Wayne	12000.00
3	3	Joker	3300.00
4	1	Billy Costigan	25000.00
5	1	Teddy Daniels	30000.00
5	6	Chuck Aule	2750.00
6	2	Agent Moreau	990.00
6	4	Hubert Bonisseur	1650.00

Autre écriture : En écrivant l'augmentation autrement

```
UPDATE jouer SET cachet = cachet + cachet * 0.10 WHERE cachet < 5000;
```

Question 10

Avec jointure : augmenter de 20 % les cachets des acteurs jouant dans les films du genre « Action » réalisés par « Nolan ».

```
UPDATE jouer j, film f, realisateur r
SET j.cachet = j.cachet * 1.20
WHERE j.idFilm = f.idFilm
  AND f.idReal = r.idReal
  AND f.genre = 'Action'
  AND r.nom = 'Nolan';
-- Vérification
SELECT f.titre, j.idActeur, j.role, j.cachet FROM jouer j, film f
WHERE j.idFilm = f.idFilm AND f.titre = 'The Dark Knight';
```

Résultat

titre	idActeur	role	cachet
The Dark Knight	2	Bruce Wayne	14400.00
The Dark Knight	3	Joker	3960.00

Autre écriture : Sans jointure, avec une sous-requête

```
UPDATE jouer
SET cachet = cachet * 1.20
WHERE idFilm IN (SELECT f.idFilm FROM film f, realisateur r
                 WHERE f.idReal = r.idReal AND f.genre = 'Action' AND r.nom = 'Nolan');
```

Question 11

Avec sous-requête : mettre `budget = 0` pour les films qui n'ont aucun acteur.

```
UPDATE film SET budget = 0 WHERE idFilm NOT IN (SELECT idFilm FROM jouer);
-- Vérification
SELECT idFilm, titre, budget FROM film;
```

Résultat

idFilm	titre	budget
1	Inception	160000000
2	Interstellar	165000000
3	The Dark Knight	185000000
4	Les Infiltrés	90000000
5	Shutter Island	80000000
6	OSS 117	14000000
7	Tenet	0

Autre écriture : Avec NOT EXISTS

```
UPDATE film f SET f.budget = 0
WHERE NOT EXISTS (SELECT * FROM jouer j WHERE j.idFilm = f.idFilm);
```

Question 12

Avec requête imbriquée : passer au genre « Blockbuster » les films dont le budget dépasse le budget moyen des films du genre « Action ». (MySQL interdit de lire la table modifiée dans la sous-requête : on passe par une table dérivée.)

```
UPDATE film
SET genre = 'Blockbuster'
WHERE budget > (SELECT AVG(f.budget)
                FROM (SELECT budget, genre FROM film) AS f
                WHERE f.genre = 'Action');
-- Vérification
SELECT idFilm, titre, genre, budget FROM film;
```

Résultat

idFilm	titre	genre	budget
1	Inception	Blockbuster	160000000
2	Interstellar	Blockbuster	165000000
3	The Dark Knight	Blockbuster	185000000
4	Les Infiltrés	Action	90000000
5	Shutter Island	Thriller	80000000
6	OSS 117	Comédie	14000000
7	Tenet	Action	0

Autre écriture : Avec une variable (moyenne calculée d'abord, puis utilisée)

```
SET @moy = (SELECT AVG(budget) FROM film WHERE genre = 'Action');
UPDATE film SET genre = 'Blockbuster' WHERE budget > @moy;
```

2.3 Suppression (DELETE) : simple, avec jointure, avec sous-requête

Question 13

Delete : Supprimer les projections ayant moins de 10 spectateurs.

```
DELETE FROM projection WHERE nbSpectateurs < 10;
-- Vérification
SELECT * FROM projection;
```

Résultat

idProj	idFilm	salle	dateProj	nbSpectateurs
1	1	Salle 1	2026-09-20	85
2	1	Salle 2	2026-10-03	120
3	2	Salle 1	2026-10-10	150
5	3	Salle 1	2026-10-24	95
6	5	Salle 2	2026-10-31	60
7	6	Salle 3	2026-11-15	70
8	2	Salle 1	2026-12-05	110

Question 14

Avec jointure : supprimer les lignes de JOUER correspondant aux films du réalisateur « Scorsese ».

```
DELETE j
FROM jouer j, film f, realisateur r
WHERE j.idFilm = f.idFilm
  AND f.idReal = r.idReal
  AND r.nom = 'Scorsese';
-- Vérification
SELECT COUNT(*) AS nb_lignes_jouer FROM jouer;
```

Résultat

nb_lignes_jouer
8

Autre écriture : Sans jointure, avec une sous-requête

```
DELETE FROM jouer
WHERE idFilm IN (SELECT f.idFilm FROM film f, realisateur r
                 WHERE f.idReal = r.idReal AND r.nom = 'Scorsese');
```

Question 15

Avec sous-requête : supprimer les acteurs qui n'ont joué dans aucun film.

```
DELETE FROM acteur WHERE idActeur NOT IN (SELECT idActeur FROM jouer);
-- Vérification
SELECT idActeur, nom, prenom FROM acteur;
```

Résultat

idActeur	nom	prenom
1	DiCaprio	Leonardo
2	Bale	Christian
3	Ledger	Heath
4	Dujardin	Jean
5	McConaughey	Matthew

Autre écriture : Avec NOT EXISTS

```
DELETE FROM acteur
WHERE NOT EXISTS (SELECT * FROM jouer j WHERE j.idActeur = acteur.idActeur);
```

Préparation de la partie 3 : jeu de données complémentaire

```
INSERT INTO acteur VALUES
(7,'Cotillard','Marion','1975-07-30','Française'),
(8,'Hathaway','Anne','1980-11-12','Américaine');
-- Vérification
SELECT * FROM acteur;
```

Résultat

idActeur	nom	prenom	dateNaiss	nationalite
1	DiCaprio	Leonardo	1974-11-11	Américaine
2	Bale	Christian	1974-11-11	Britannique
3	Ledger	Heath	1979-04-04	Australienne
4	Dujardin	Jean	1972-06-19	Française
5	McConaughey	Matthew	1969-11-04	Américaine
7	Cotillard	Marion	1975-07-30	Française
8	Hathaway	Anne	1980-11-12	Américaine

Partie 3 : Select : projection, sélection, jointure, regroupement, auto-jointure, imbrication

3.1 Projection et sélection (une seule table) : WHERE, BETWEEN, IN, LIKE, IS NULL, DISTINCT, tri, LIMIT

Question 16

Projection et alias : Afficher le titre et la durée de chaque film, la durée étant renommée « dureeMin ».

```
SELECT titre, duree AS dureeMin FROM film;
```

Résultat

titre	dureeMin
Inception	148
Interstellar	169
The Dark Knight	152
Les Infiltrés	151
Shutter Island	138
OSS 117	99
Tenet	*NULL*

Autre écriture : Alias sans le mot AS

```
SELECT titre, duree dureeMin FROM film;
```

Question 17

WHERE : Afficher les films du genre « Action ».

```
SELECT idFilm, titre, genre FROM film WHERE genre = 'Action';
```

Résultat

idFilm	titre	genre
4	Les Infiltrés	Action
7	Tenet	Action

Question 18

BETWEEN : Afficher le titre et l'année des films sortis entre 2008 et 2010 (bornes comprises), par année croissante.

```
SELECT titre, annee FROM film WHERE annee BETWEEN 2008 AND 2010 ORDER BY annee, titre;
```

Résultat

titre	annee
The Dark Knight	2008
Inception	2010
Shutter Island	2010

Autre écriture : Avec les opérateurs >= et <=

```
SELECT titre, annee FROM film WHERE annee >= 2008 AND annee <= 2010 ORDER BY annee, titre;
```

Question 19

IN : Afficher les films dont le genre est « Thriller » ou « Comédie ».

```
SELECT titre, genre FROM film WHERE genre IN ('Thriller', 'Comédie');
```

Résultat

titre	genre
Shutter Island	Thriller
OSS 117	Comédie

Autre écriture : Avec OR

```
SELECT titre, genre FROM film WHERE genre = 'Thriller' OR genre = 'Comédie';
```

Question 20

LIKE : Afficher les films dont le titre commence par la lettre « I ».

```
SELECT titre FROM film WHERE titre LIKE 'I%';
```

Résultat

titre
Inception
Interstellar

% remplace n'importe quelle suite de caractères (même vide) ; _ remplace exactement un caractère. « Les Infiltrés » n'est pas affiché : son titre commence par « L ».

Autre écriture : Avec LEFT

```
SELECT titre FROM film WHERE LEFT(titre, 1) = 'I';
```

Autre écriture : Avec SUBSTRING

```
SELECT titre FROM film WHERE SUBSTRING(titre, 1, 1) = 'I';
```

Question 21

IS NULL : Afficher les films dont l'année de sortie n'est pas renseignée.

```
SELECT idFilm, titre FROM film WHERE annee IS NULL;
```

Résultat

idFilm	titre
7	Tenet

Piège : `WHERE annee = NULL` n'affiche jamais rien, car comparer à NULL donne UNKNOWN. Il faut toujours écrire `IS NULL` (ou `IS NOT NULL`).

Question 22

AND et négation : Afficher les films de plus de 140 minutes dont le genre n'est pas « Blockbuster ».

```
SELECT titre, genre, duree FROM film WHERE duree > 140 AND genre <> 'Blockbuster';
```

Résultat

titre	genre	duree
Les Infiltrés	Action	151

Autre écriture : Avec NOT

```
SELECT titre, genre, duree FROM film WHERE duree > 140 AND NOT (genre = 'Blockbuster');
```

Autre écriture : Avec !=

```
SELECT titre, genre, duree FROM film WHERE duree > 140 AND genre != 'Blockbuster';
```

Question 23

DISTINCT : Afficher la liste des genres de films, sans doublon.

```
SELECT DISTINCT genre FROM film;
```

Résultat

genre
Blockbuster
Action
Thriller
Comédie

Autre écriture : Avec GROUP BY

```
SELECT genre FROM film GROUP BY genre;
```

Question 24

Tri sur une colonne texte : Afficher le nom et le prénom des acteurs de nationalité américaine, triés par nom.

```
SELECT nom, prenom FROM acteur WHERE nationalite = 'Américaine' ORDER BY nom;
```

Résultat

nom	prenom
DiCaprio	Leonardo
Hathaway	Anne
McConaughey	Matthew

Question 25

Tri multicritère : Afficher les films triés par année (ordre décroissant), puis par titre (ordre croissant).

```
SELECT * FROM film ORDER BY annee DESC, titre ASC;
```

Résultat

idFilm	titre	genre	annee	duree	budget	idReal
2	Interstellar	Blockbuster	2014	169	165000000	1
1	Inception	Blockbuster	2010	148	160000000	1
5	Shutter Island	Thriller	2010	138	80000000	2
3	The Dark Knight	Blockbuster	2008	152	185000000	1
4	Les Infiltrés	Action	2006	151	90000000	2
6	OSS 117	Comédie	2006	99	14000000	3
7	Tenet	Action	*NULL*	*NULL*	0	1

Les valeurs NULL (ici l'année de « Tenet ») sont placées en dernier avec DESC dans MariaDB.

Question 26

ORDER BY et LIMIT : Afficher les 3 films les plus longs.

```
SELECT titre, duree FROM film ORDER BY duree DESC LIMIT 3;
```

Résultat

titre	duree
Interstellar	169
The Dark Knight	152
Les Infiltrés	151

Autre écriture : Avec LIMIT décalage, nombre

```
SELECT titre, duree FROM film ORDER BY duree DESC LIMIT 0, 3;
```

3.2 Jointure de plusieurs tables : table1.x = table2.x

Question 27

Jointure : Afficher le titre de chaque film avec le nom et le prénom de son réalisateur.

```
SELECT f.titre, r.nom, r.prenom
FROM film f, realisateur r
WHERE f.idReal = r.idReal;
```

Résultat

titre	nom	prenom
Inception	Nolan	Christopher
Interstellar	Nolan	Christopher
The Dark Knight	Nolan	Christopher
Les Infiltrés	Scorsese	Martin
Shutter Island	Scorsese	Martin
OSS 117	Hazanavicius	Michel
Tenet	Nolan	Christopher

Question 28

Afficher le nom de l'acteur, le titre du film, le rôle et le cachet.

```
SELECT a.nom, f.titre, j.role, j.cachet
FROM acteur a, film f, jouer j
WHERE j.idActeur = a.idActeur AND j.idFilm = f.idFilm
ORDER BY f.titre, j.cachet DESC;
```

Résultat

nom	titre	role	cachet
DiCaprio	Inception	Dom Cobb	20000.00
Ledger	Inception	Arthur	4950.00
Dujardin	Inception	Eames	4400.00
McConaughey	Interstellar	Cooper	15000.00
Dujardin	OSS 117	Hubert Bonisseur	1650.00
Bale	OSS 117	Agent Moreau	990.00
Bale	The Dark Knight	Bruce Wayne	14400.00
Ledger	The Dark Knight	Joker	3960.00

Question 29

Afficher les acteurs ayant joué dans un film de « Nolan » avec un cachet supérieur à 10000.

```
SELECT DISTINCT a.nom, a.prenom
FROM acteur a, jouer j, film f, realisateur r
WHERE j.idActeur = a.idActeur AND j.idFilm = f.idFilm AND f.idReal = r.idReal
  AND r.nom = 'Nolan' AND j.cachet > 10000;
```

Résultat

nom	prenom
DiCaprio	Leonardo
McConaughey	Matthew
Bale	Christian

Autre écriture : Avec des sous-requêtes imbriquées

```
SELECT nom, prenom
FROM acteur
WHERE idActeur IN (SELECT j.idActeur FROM jouer j, film f
                   WHERE j.idFilm = f.idFilm
                     AND j.cachet > 10000
                     AND f.idReal = (SELECT idReal FROM realisateur WHERE nom = 'Nolan'));
```

3.3 Regroupement : fonctions d'agrégation, GROUP BY, HAVING

Question 30

Group by / Having : Afficher le nombre de films par genre.

```
SELECT genre, COUNT(*) AS nbFilms
FROM film
GROUP BY genre;
```

Résultat

genre	nbFilms
Action	2
Blockbuster	3
Comédie	1
Thriller	1

Question 31

Afficher le cachet moyen par film.

```
SELECT f.titre, ROUND(AVG(j.cachet), 2) AS cachetMoyen
FROM film f, jouer j
WHERE f.idFilm = j.idFilm
GROUP BY f.idFilm, f.titre;
```

Résultat

titre	cachetMoyen
Inception	9783.33
Interstellar	15000.00
The Dark Knight	9180.00
OSS 117	1320.00

Question 32

Afficher les réalisateurs ayant réalisé plus de 3 films.

```
SELECT r.nom, r.prenom, COUNT(*) AS nbFilms
FROM realisateur r, film f
WHERE r.idReal = f.idReal
GROUP BY r.idReal, r.nom, r.prenom
HAVING COUNT(*) > 3;
```

Résultat

nom	prenom	nbFilms
Nolan	Christopher	4

Autre écriture : Avec une sous-requête dans FROM

```
SELECT nom, prenom, nbFilms
FROM (SELECT r.nom, r.prenom, COUNT(*) AS nbFilms
      FROM realisateur r, film f
      WHERE r.idReal = f.idReal
      GROUP BY r.idReal, r.nom, r.prenom) AS t
WHERE nbFilms > 3;
```

Question 33

Afficher les acteurs dont le cachet total est supérieur à 15000 (Having). (Le seuil de 15000 est adapté au jeu de test réduit.)

```
SELECT a.nom, a.prenom, SUM(j.cachet) AS cachetTotal
FROM acteur a, jouer j
WHERE a.idActeur = j.idActeur
GROUP BY a.idActeur, a.nom, a.prenom
HAVING SUM(j.cachet) > 15000;
```

Résultat

nom	prenom	cachetTotal
DiCaprio	Leonardo	20000.00
Bale	Christian	15390.00

McConaughey (15000 exactement) n'est pas affiché : la condition est « strictement supérieur ».

Autre écriture : Avec une sous-requête dans FROM

```
SELECT a.nom, a.prenom, t.cachetTotal
FROM acteur a,
     (SELECT idActeur, SUM(cachet) AS cachetTotal
      FROM jouer GROUP BY idActeur HAVING SUM(cachet) > 15000) AS t
WHERE a.idActeur = t.idActeur;
```

Question 34

Afficher, par genre et par réalisateur, le budget maximal, à condition qu'il dépasse 100000000.

```
SELECT f.genre, r.nom, MAX(f.budget) AS budgetMax
FROM film f, realisateur r
WHERE f.idReal = r.idReal
GROUP BY f.genre, r.nom
HAVING MAX(f.budget) > 100000000;
```

Résultat

genre	nom	budgetMax
Blockbuster	Nolan	185000000

3.4 Auto-jointure : une table jointe avec elle-même

Question 35

Auto-requête (auto-jointure) : Ajouter à ACTEUR la colonne `idParrain` (l'acteur qui a introduit le débutant dans le métier). Afficher chaque acteur avec le nom de son parrain.

```
ALTER TABLE acteur ADD idParrain INT, ADD FOREIGN KEY (idParrain) REFERENCES acteur(idActeur);
UPDATE acteur SET idParrain = 1 WHERE idActeur IN (2, 5);
UPDATE acteur SET idParrain = 2 WHERE idActeur = 3;
SELECT a.nom AS acteur, p.nom AS parrain
FROM acteur a, acteur p
WHERE a.idParrain = p.idActeur;
```

Résultat

acteur	parrain
Bale	DiCaprio
Ledger	Bale
McConaughey	DiCaprio

Question 36

Afficher les couples de films du même réalisateur (sans doublons ni couple (x, x)).

```
SELECT f1.titre AS film1, f2.titre AS film2, r.nom AS realisateur
FROM film f1, film f2, realisateur r
WHERE f1.idReal = f2.idReal
  AND f1.idFilm < f2.idFilm
  AND f1.idReal = r.idReal;
```

Résultat

film1	film2	realisateur
Inception	Interstellar	Nolan
Inception	The Dark Knight	Nolan
Inception	Tenet	Nolan
Interstellar	The Dark Knight	Nolan
Interstellar	Tenet	Nolan
The Dark Knight	Tenet	Nolan
Les Infiltrés	Shutter Island	Scorsese

Question 37

Afficher les acteurs ayant la même date de naissance qu'un autre acteur.

```
SELECT DISTINCT a1.nom, a1.prenom, a1.dateNaiss
FROM acteur a1, acteur a2
WHERE a1.dateNaiss = a2.dateNaiss AND a1.idActeur <> a2.idActeur;
```

Résultat

nom	prenom	dateNaiss
Bale	Christian	1974-11-11
DiCaprio	Leonardo	1974-11-11

Autre écriture : Avec EXISTS

```
SELECT nom, prenom, dateNaiss
FROM acteur a1
WHERE EXISTS (SELECT * FROM acteur a2
              WHERE a2.dateNaiss = a1.dateNaiss AND a2.idActeur <> a1.idActeur);
```

Autre écriture : Avec IN et GROUP BY ... HAVING

```
SELECT nom, prenom, dateNaiss
FROM acteur
WHERE dateNaiss IN (SELECT dateNaiss FROM acteur GROUP BY dateNaiss HAVING COUNT(*) > 1);
```

3.5 Requêtes imbriquées : sous-requêtes avec IN, EXISTS, NOT IN, comparaison et table dérivée

Question 38

Requêtes imbriquées : Afficher les films dont la durée dépasse la durée moyenne de tous les films.

```
SELECT titre, duree
FROM film
WHERE duree > (SELECT AVG(duree) FROM film);
```

Résultat

titre	duree
Inception	148
Interstellar	169
The Dark Knight	152
Les Infiltrés	151

Autre écriture : Avec une sous-requête dans FROM

```
SELECT f.titre, f.duree
FROM film f, (SELECT AVG(duree) AS dureeMoyenne FROM film) AS t
WHERE f.duree > t.dureeMoyenne;
```

Question 39

Afficher les acteurs ayant joué dans au moins un film du genre « Comédie » (IN).

```
SELECT nom, prenom
FROM acteur
WHERE idActeur IN (SELECT j.idActeur FROM jouer j, film f
                   WHERE j.idFilm = f.idFilm AND f.genre = 'Comédie');
```

Résultat

nom	prenom
Bale	Christian
Dujardin	Jean

Autre écriture : Avec une jointure et DISTINCT

```
SELECT DISTINCT a.nom, a.prenom
FROM acteur a, jouer j, film f
WHERE a.idActeur = j.idActeur AND j.idFilm = f.idFilm AND f.genre = 'Comédie';
```

Autre écriture : Avec EXISTS

```
SELECT nom, prenom
FROM acteur a
WHERE EXISTS (SELECT * FROM jouer j, film f
              WHERE j.idActeur = a.idActeur AND j.idFilm = f.idFilm AND f.genre = 'Comédie');
```

Question 40

Afficher les acteurs n'ayant joué dans aucun film (NOT IN ou NOT EXISTS).

```
SELECT nom, prenom
FROM acteur a
WHERE NOT EXISTS (SELECT * FROM jouer j WHERE j.idActeur = a.idActeur);
```

Résultat

nom	prenom
Cotillard	Marion
Hathaway	Anne

Autre écriture : Avec NOT IN

```
SELECT nom, prenom
FROM acteur
WHERE idActeur NOT IN (SELECT idActeur FROM jouer);
```

Question 41

Afficher l'acteur ayant le cachet le plus élevé dans le film « Inception ».

```
SELECT a.nom, a.prenom, j.cachet
FROM acteur a, jouer j, film f
WHERE a.idActeur = j.idActeur AND j.idFilm = f.idFilm
  AND f.titre = 'Inception'
  AND j.cachet = (SELECT MAX(j2.cachet) FROM jouer j2, film f2
                  WHERE j2.idFilm = f2.idFilm AND f2.titre = 'Inception');
```

Résultat

nom	prenom	cachet
DiCaprio	Leonardo	20000.00

Autre écriture : Avec ORDER BY ... LIMIT 1 (attention : un seul acteur affiché en cas d'égalité)

```
SELECT a.nom, a.prenom, j.cachet
FROM acteur a, jouer j, film f
WHERE a.idActeur = j.idActeur AND j.idFilm = f.idFilm AND f.titre = 'Inception'
ORDER BY j.cachet DESC
LIMIT 1;
```

Question 42

Afficher les films dont le nombre d'acteurs est supérieur au nombre moyen d'acteurs par film.

```
SELECT f.titre, COUNT(*) AS nbActeurs
FROM film f, jouer j
WHERE f.idFilm = j.idFilm
GROUP BY f.idFilm, f.titre
HAVING COUNT(*) > (SELECT AVG(nb) FROM (SELECT COUNT(*) AS nb FROM jouer GROUP BY idFilm) AS t);
```

Résultat

titre	nbActeurs
Inception	3

Autre écriture : En calculant le nombre moyen d'acteurs sans table dérivée

```
SELECT f.titre, COUNT(*) AS nbActeurs
FROM film f, jouer j
WHERE f.idFilm = j.idFilm
GROUP BY f.idFilm, f.titre
HAVING COUNT(*) > (SELECT COUNT(*) FROM jouer) / (SELECT COUNT(DISTINCT idFilm) FROM jouer);
```

Partie 4 : Requêtes sur les dates : YEAR, MONTH, TIMESTAMPDIFF, CURDATE, DATE_SUB, DAYNAME

Question 43

Afficher les acteurs nés en 1980.

```
SELECT nom, prenom, dateNaiss FROM acteur WHERE YEAR(dateNaiss) = 1980;
```

Résultat

nom	prenom	dateNaiss
Hathaway	Anne	1980-11-12

Autre écriture : Avec BETWEEN

```
SELECT nom, prenom, dateNaiss
FROM acteur
WHERE dateNaiss BETWEEN '1980-01-01' AND '1980-12-31';
```

Autre écriture : Avec LIKE (possible, mais déconseillé : la date est convertie en texte)

```
SELECT nom, prenom, dateNaiss
FROM acteur
WHERE dateNaiss LIKE '1980%';
```

Question 44

Afficher le nom et l'âge de chaque acteur (`TIMESTAMPDIFF`).

```
SELECT nom, TIMESTAMPDIFF(YEAR, dateNaiss, CURDATE()) AS age FROM acteur;
```

Résultat

nom	age
DiCaprio	51
Bale	51
Ledger	47
Dujardin	54
McConaughey	56
Cotillard	51
Hathaway	45

Autre écriture : Formule exacte avec YEAR et DATE_FORMAT

```
SELECT nom,
       (YEAR(CURDATE()) - YEAR(dateNaiss))
       - (DATE_FORMAT(CURDATE(), '%m%d') < DATE_FORMAT(dateNaiss, '%m%d')) AS age
FROM acteur;
```

Question 45

Afficher les projections programmées entre le 01/11/2026 et le 31/12/2026.

```
SELECT f.titre, p.salle, p.dateProj
FROM film f, projection p
WHERE f.idFilm = p.idFilm
  AND p.dateProj BETWEEN '2026-11-01' AND '2026-12-31'
ORDER BY p.dateProj;
```

Résultat

titre	salle	dateProj
OSS 117	Salle 3	2026-11-15
Interstellar	Salle 1	2026-12-05

Autre écriture : Avec des comparaisons >= et <=

```
SELECT f.titre, p.salle, p.dateProj
FROM film f, projection p
WHERE f.idFilm = p.idFilm
  AND p.dateProj >= '2026-11-01' AND p.dateProj <= '2026-12-31'
ORDER BY p.dateProj;
```

Question 46

Afficher le nombre de projections par mois (`MONTH`, `GROUP BY`).

```
SELECT MONTH(dateProj) AS mois, COUNT(*) AS nbProjections
FROM projection
GROUP BY MONTH(dateProj);
```

Résultat

mois	nbProjections
9	1
10	4
11	1
12	1

Autre écriture : Avec un alias dans GROUP BY

```
SELECT MONTH(dateProj) AS mois, COUNT(*) AS nbProjections
FROM projection
GROUP BY mois
ORDER BY mois;
```

Question 47

Afficher les acteurs nés un mois de juillet.

```
SELECT nom, prenom, dateNaiss FROM acteur WHERE MONTH(dateNaiss) = 7;
```

Résultat

nom	prenom	dateNaiss
Cotillard	Marion	1975-07-30

Autre écriture : Avec DATE_FORMAT

```
SELECT nom, prenom, dateNaiss FROM acteur WHERE DATE_FORMAT(dateNaiss, '%m') = '07';
```

Autre écriture : Avec LIKE (déconseillé)

```
SELECT nom, prenom, dateNaiss FROM acteur WHERE dateNaiss LIKE '%-07-%';
```

Question 48

Afficher les projections des 30 derniers jours (`CURDATE()`, `DATE_SUB`).

```
SELECT f.titre, p.dateProj
FROM film f, projection p
WHERE f.idFilm = p.idFilm
  AND p.dateProj BETWEEN DATE_SUB(CURDATE(), INTERVAL 30 DAY) AND CURDATE()
ORDER BY p.dateProj;
```

Résultat

titre	dateProj
Inception	2026-10-03
Interstellar	2026-10-10
The Dark Knight	2026-10-24
Shutter Island	2026-10-31

Autre écriture : Avec CURDATE() - INTERVAL

```
SELECT f.titre, p.dateProj
FROM film f, projection p
WHERE f.idFilm = p.idFilm
  AND p.dateProj >= CURDATE() - INTERVAL 30 DAY AND p.dateProj <= CURDATE()
ORDER BY p.dateProj;
```

Autre écriture : Avec DATEDIFF

```
SELECT f.titre, p.dateProj
FROM film f, projection p
WHERE f.idFilm = p.idFilm AND DATEDIFF(CURDATE(), p.dateProj) BETWEEN 0 AND 30
ORDER BY p.dateProj;
```

Question 49

Afficher le jour de la semaine de chaque projection (`DAYNAME`). Pour obtenir les noms de jours en français, il faut d'abord régler la langue de la session.

```
SET lc_time_names = 'fr_FR';
SELECT f.titre, p.dateProj, DAYNAME(p.dateProj) AS jour
FROM film f, projection p
WHERE f.idFilm = p.idFilm
ORDER BY p.dateProj;
```

Résultat

titre	dateProj	jour
Inception	2026-09-20	dimanche
Inception	2026-10-03	samedi
Interstellar	2026-10-10	samedi
The Dark Knight	2026-10-24	samedi
Shutter Island	2026-10-31	samedi
OSS 117	2026-11-15	dimanche
Interstellar	2026-12-05	samedi

Sans la ligne `SET lc_time_names`, MariaDB affiche les jours en anglais (Sunday, Saturday...).

Autre écriture : Avec DATE_FORMAT (%W = nom du jour)

```
SET lc_time_names = 'fr_FR';
SELECT f.titre, p.dateProj, DATE_FORMAT(p.dateProj, '%W') AS jour
FROM film f, projection p
WHERE f.idFilm = p.idFilm
ORDER BY p.dateProj;
```

Partie 5 (Complément) : ALTER TABLE, DEFAULT, NOT NULL et clés étrangères en cascade

Cette partie complète les questions précédentes. Les instructions sont à exécuter dans l'ordre, sur la même base cinema.

5.1 DEFAULT

Question 50 — Ajouter une colonne avec DEFAULT

Ajouter à FILM une colonne `paysOrigine` (VARCHAR(30)) obligatoire, avec la valeur par défaut « Inconnue ». La table contient déjà des films. Afficher l'identifiant, le titre et le pays d'origine.

```
ALTER TABLE film
ADD paysOrigine VARCHAR(30) NOT NULL DEFAULT 'Inconnue';

SELECT idFilm, titre, paysOrigine
FROM film
ORDER BY idFilm;
```

Résultat

idFilm	titre	paysOrigine
1	Inception	Inconnue
2	Interstellar	Inconnue
3	The Dark Knight	Inconnue
4	Les Infiltrés	Inconnue
5	Shutter Island	Inconnue
6	OSS 117	Inconnue
7	Tenet	Inconnue

À retenir : `DEFAULT` fournit une valeur lorsqu'aucune valeur n'est donnée à l'insertion. Lorsqu'une colonne avec `DEFAULT` est ajoutée à une table déjà remplie, les lignes existantes prennent cette valeur par défaut.

Question 51 — Modifier la valeur DEFAULT

Remplacer la valeur par défaut de `paysOrigine` par « Tunisie », puis insérer un film sans préciser cette colonne. Afficher les deux films concernés.

```
ALTER TABLE film
ALTER paysOrigine SET DEFAULT 'Tunisie';

INSERT INTO film (titre, genre, idReal)
VALUES ('Film test', 'Drame', 1);

SELECT titre, paysOrigine
FROM film
WHERE titre IN ('Inception', 'Film test')
ORDER BY titre;
```

Résultat

titre	paysOrigine
Film test	Tunisie
Inception	Inconnue

Résultat attendu : modifier `DEFAULT` ne remplace pas les valeurs déjà enregistrées. La nouvelle valeur s'applique aux insertions futures qui omettent la colonne.

5.2 NOT NULL sur une table déjà remplie

Question 52 — ALTER TABLE refusé à cause de données déjà présentes

On veut rendre `nationalite` obligatoire dans ACTEUR. Pour observer le problème, placer volontairement NULL dans une ligne, puis tenter la modification.

```
UPDATE acteur
SET nationalite = NULL
WHERE idActeur = 8;

ALTER TABLE acteur
MODIFY nationalite VARCHAR(30) NOT NULL;
```

Résultat

```
Erreur 1265 : Data truncated for column 'nationalite' at row 7
```

Résultat / explication : la modification échoue car la colonne contient déjà une valeur NULL. MariaDB ne peut pas transformer cette valeur en une valeur obligatoire sans qu'on fournisse d'abord une valeur valide. Message obtenu : `Erreur 1265 : Data truncated for column 'nationalite' at row 7`

Corriger les données, puis relancer `ALTER TABLE` :

```
UPDATE acteur
SET nationalite = 'Non precisee'
WHERE nationalite IS NULL;

ALTER TABLE acteur
MODIFY nationalite VARCHAR(30) NOT NULL;

SELECT idActeur, nom, nationalite
FROM acteur
WHERE idActeur = 8;
```

Résultat

idActeur	nom	nationalite
8	Hathaway	Non precisee

Conclusion : avant de rendre une colonne NOT NULL, il faut vérifier qu'aucune ligne ne contient NULL. On peut corriger les anciennes lignes avec `UPDATE`, puis appliquer `ALTER TABLE`.

5.3 Clés étrangères en cascade : ON UPDATE CASCADE, ON DELETE CASCADE

Question 53 — ON UPDATE CASCADE

Créer une table de test qui référence FILM. La clé étrangère doit propager automatiquement la modification de `idFilm`. Insérer deux projections, modifier l'identifiant du film 7 en 70, puis afficher la table.

```
CREATE TABLE projection_test (
  idProj INT PRIMARY KEY,
  idFilm INT,
  salle  VARCHAR(20),
  CONSTRAINT fk_test_film
    FOREIGN KEY (idFilm) REFERENCES film(idFilm)
    ON UPDATE CASCADE
    ON DELETE CASCADE
);

INSERT INTO projection_test VALUES
(101, 7, 'Salle A'),
(102, 1, 'Salle B');

UPDATE film
SET idFilm = 70
WHERE idFilm = 7;

SELECT * FROM projection_test
ORDER BY idProj;
```

Résultat

idProj	idFilm	salle
101	70	Salle A
102	1	Salle B

Résultat attendu : la ligne de projection qui référençait le film 7 référence maintenant le film 70, sans UPDATE direct sur `projection_test`. `ON UPDATE CASCADE` signifie : si la valeur de la clé référencée change dans la table parent FILM, MariaDB reporte automatiquement cette modification dans la clé étrangère de la table enfant.

Question 54 — ON DELETE CASCADE

Supprimer le film dont l'identifiant est 70, puis afficher le contenu de `projection_test`.

```
DELETE FROM film
WHERE idFilm = 70;

SELECT * FROM projection_test
ORDER BY idProj;
```

Résultat

idProj	idFilm	salle
102	1	Salle B

Résultat attendu : la projection 101 disparaît automatiquement car elle dépendait du film 70. La projection 102 reste présente. `ON DELETE CASCADE` signifie : lorsqu'une ligne parent est supprimée dans FILM, les lignes enfants qui la référencent sont supprimées automatiquement. **Attention :** cette action peut supprimer plusieurs lignes liées.

Question 55 — Prévoir le résultat d'une requête

Après les opérations précédentes, exécuter la requête suivante. Indiquer les lignes affichées.

```
SELECT p.idProj, p.idFilm, f.titre
FROM projection_test p, film f
WHERE p.idFilm = f.idFilm
ORDER BY p.idProj;
```

Résultat

idProj	idFilm	titre
102	1	Inception

Résultat attendu : une seule ligne. La projection 101 a été supprimée par `ON DELETE CASCADE` ; la projection 102 référence toujours Inception.

Partie 6 : Prévoir le résultat : agrégation, NULL, jointure, regroupement, auto-jointure, dates

Pour chaque requête, indiquer les lignes affichées, sans l'exécuter.

Question 56 — Fonction d'agrégation

```
SELECT COUNT(*) AS nb FROM jouer WHERE cachet > 10000;
```

Résultat

nb
3

Dom Cobb 20000, Cooper 15000, Bruce Wayne 14400.

Question 57 — COUNT(*) et COUNT(colonne)

```
SELECT COUNT(*) AS nbFilms, COUNT(duree) AS nbDurees FROM film;
```

Résultat

nbFilms	nbDurees
7	6

`COUNT(colonne)` ignore les NULL : le film « Film test » n'a pas de durée.

Question 58 — AVG et NULL

```
SELECT AVG(duree) AS moyenne FROM film;
```

Résultat

moyenne
142.8333

857 / 6 : le NULL n'entre ni dans la somme ni dans le nombre de lignes.

Question 59 — Piège du NULL avec <>

```
SELECT titre FROM film WHERE duree <> 148;
```

Résultat

titre
Interstellar
The Dark Knight
Les Infiltrés
Shutter Island
OSS 117

« Film test » n'apparaît pas : `NULL <> 148` n'est ni vrai ni faux (UNKNOWN).

Question 60 — GROUP BY sur une jointure

```
SELECT a.nom, COUNT(*) AS nbFilms
FROM acteur a, jouer j
WHERE a.idActeur = j.idActeur
GROUP BY a.idActeur, a.nom;
```

Résultat

nom	nbFilms
DiCaprio	1
Bale	2
Ledger	2
Dujardin	2
McConaughey	1

Cotillard et Hathaway sont absentes : la jointure élimine les acteurs sans ligne dans `jouer`.

Question 61 — HAVING

```
SELECT f.titre, SUM(p.nbSpectateurs) AS total
FROM film f, projection p
WHERE f.idFilm = p.idFilm
GROUP BY f.idFilm, f.titre
HAVING SUM(p.nbSpectateurs) > 150;
```

Résultat

titre	total
Inception	205
Interstellar	260

Inception : 85 + 120 = 205 ; Interstellar : 150 + 110 = 260. The Dark Knight n'a plus que 95 spectateurs (la projection de 8 a été supprimée).

Question 62 — Statistiques par salle

```
SELECT salle, COUNT(*) AS nbProj, MAX(nbSpectateurs) AS maxSpect
FROM projection
GROUP BY salle;
```

Résultat

salle	nbProj	maxSpect
Salle 1	4	150
Salle 2	2	120
Salle 3	1	70

Question 63 — Auto-jointure avec condition supplémentaire

```
SELECT a.nom AS acteur, p.nom AS parrain
FROM acteur a, acteur p
WHERE a.idParrain = p.idActeur AND p.idParrain IS NULL;
```

Résultat

acteur	parrain
Bale	DiCaprio
McConaughey	DiCaprio

Ledger a pour parrain Bale, qui a lui-même un parrain : la ligne est exclue.

Question 64 — Dates

```
SELECT nom, dateNaiss FROM acteur WHERE MONTH(dateNaiss) = 11;
```

Résultat

nom	dateNaiss
DiCaprio	1974-11-11
Bale	1974-11-11
McConaughey	1969-11-04
Hathaway	1980-11-12

Question 65 — Somme sur un film

```
SELECT SUM(j.cachet) AS total
FROM jouer j, film f
WHERE j.idFilm = f.idFilm AND f.titre = 'Inception';
```

Résultat

total
29350.00

20000 + 4950 + 4400.

Partie 7 : Prévoir l'erreur : clé primaire, CHECK, clés étrangères

Pour chaque instruction, dire si elle réussit ou échoue, et expliquer pourquoi. (Les messages affichés sont ceux de MariaDB, en anglais.)

Question 66 — Clé primaire en double

```
INSERT INTO jouer VALUES (1, 1, 'Figurant', 100);
```

Résultat

```
Erreur 1062 : Duplicate entry '1-1' for key 'PRIMARY'
```

Échec : la clé primaire (idFilm, idActeur) = (1, 1) existe déjà. Message : `Erreur 1062 : Duplicate entry '1-1' for key 'PRIMARY'`

Question 67 — Contrainte CHECK

```
INSERT INTO film (titre, genre, annee)
VALUES ('Metropolis', 'Drame', 1850);
```

Résultat

```
Erreur 4025 : CONSTRAINT `film.annee` failed for `cinema`.`film`
```

Échec : la contrainte `CHECK (annee >= 1900)` est violée. Message : `Erreur 4025 : CONSTRAINT film.annee failed for cinema.film`

Question 68 — Clé étrangère inexistante

```
INSERT INTO projection
VALUES (10, 99, 'Salle 1', '2026-11-20', 50);
```

Résultat

```
Erreur 1452 : Cannot add or update a child row: a foreign key constraint fails
(`cinema`.`projection`, CONSTRAINT `projection_ibfk_1` FOREIGN KEY (`idFilm`) REFERENCES `film`
(`idFilm`))
```

Échec : le film 99 n'existe pas, la clé étrangère `idFilm` est violée. Message : `Erreur 1452 : Cannot add or update a child row: a foreign key constraint fails (cinema.projection, CONSTRAINT projection_ibfk_1 FOREIGN KEY (idFilm) REFERENCES film (idFilm))`

Question 69 — Suppression d'une ligne parent

```
DELETE FROM realisateur WHERE idReal = 3;
```

Résultat

```
Erreur 1451 : Cannot delete or update a parent row: a foreign key constraint fails
(`cinema`.`film`, CONSTRAINT `film_ibfk_1` FOREIGN KEY (`idReal`) REFERENCES `realisateur`
(`idReal`))
```

Échec : le film « OSS 117 » référence ce réalisateur et la clé étrangère n'a pas de `ON DELETE CASCADE`. Message : `Erreur 1451 : Cannot delete or update a parent row: a foreign key constraint fails (cinema.film, CONSTRAINT film_ibfk_1 FOREIGN KEY (idReal) REFERENCES realisateur (idReal))`

Question 70 — Suppression sans conflit

```
DELETE FROM film WHERE idFilm = 8;
SELECT COUNT(*) AS nb FROM film;
```

Résultat

nb
6

Réussit : le film 8 (« Film test ») n'est référencé par aucune autre table, donc aucune contrainte n'est violée.

Synthèse à retenir

- **DEFAULT :** valeur utilisée lorsqu'une insertion ne fournit pas de valeur pour la colonne.
- **Modifier DEFAULT :** change la valeur appliquée aux prochaines insertions ; ne modifie pas les anciennes lignes.
- **MODIFY :** redéfinit toute la colonne : il faut répéter `NOT NULL` si on veut le conserver.
- **NOT NULL :** impossible si des lignes contiennent encore NULL ; corriger les données avant `ALTER TABLE`.
- **NULL :** se teste avec `IS NULL` ; `= NULL` et `<> valeur` ne donnent jamais les lignes NULL.
- **ON UPDATE CASCADE :** propage la modification de la clé parent vers les clés étrangères enfants.
- **ON DELETE CASCADE :** supprime automatiquement les lignes enfants liées à la ligne parent supprimée.

Remarque pédagogique : la table `projection_test` et les opérations sur le film 7 sont prévues pour cette démonstration. Elles servent à observer les cascades sans modifier la structure initiale de PROJECTION.